GUIA TÉCNICO | 03

Preparação de dados para redes neurais

Capítulo 03/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Separar treino/teste antes de ajustar transformadores e impedir vazamento de informação.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

PROVENIÊNCIA E PRÉ-REQUISITOS

- Material histórico de apoio: Unidade 2 — Preliminares II, páginas 81–125 do PDF consolidado.
-

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Codificações, imputação e escalonamento devem aprender parâmetros exclusivamente no treino. Redes neurais são sensíveis à escala e à escolha da função de perda. Tratar classes e variáveis temporais de modo apropriado.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: z=(x-μ _treino)/σ _treino

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Este experimento ilustra a ordem correta das operações: divisão antes de ajuste, transformação aprendida apenas no treino, baseline e avaliação em dados não vistos. O dataset Breast Cancer está integrado ao scikit-learn, sem download. O experimento usa regressão logística como referência de preparação de dados; não representa treinamento de rede neural.
Análise e exercícios

Interpretação: o scaler está dentro do pipeline e aprende estatísticas só de X_treino. O teste é usado apenas para avaliação. O conjunto é didático; acurácia e acurácia balanceada não substituem avaliação externa, análise de risco ou validação clínica. Atividade: use uma validação cruzada estratificada *somente no treino* para comparar duas configurações e preserve X_teste até o final.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X,y=load_breast_cancer(return_X_y=True)
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=.2,random_state=42,stratify=y)
s=StandardScaler().fit(Xtr)
print('Treino/teste:',Xtr.shape,Xte.shape,'média
    treino=',round(s.transform(Xtr).mean(),5))
assert Xtr.shape[0]+Xte.shape[0]==len(X)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Experimento completo — pipeline, baseline e avaliação sem vazamento

Este experimento ilustra a ordem correta das operações: divisão antes de ajuste, transformação aprendida apenas no treino, baseline e avaliação em dados não vistos. O dataset Breast Cancer está integrado ao scikit-learn, sem download. O experimento usa regressão logística como referência de preparação de dados; não representa treinamento de rede neural.
Análise e exercícios

Interpretação: o scaler está dentro do pipeline e aprende estatísticas só de X_treino. O teste é usado apenas para avaliação. O conjunto é didático; acurácia e acurácia balanceada não substituem avaliação externa, análise de risco ou validação clínica. Atividade: use uma validação cruzada estratificada *somente no treino* para comparar duas configurações e preserve X_teste até o final.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, balanced_accuracy_score, confusion_matrix
```

```
X, y = load_breast_cancer(return_X_y=True)
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
modelo = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000,  ↪
    random_state=42))
modelo.fit(X_treino, y_treino)
```

```
previsoes = modelo.predict(X_teste)
baseline = DummyClassifier(strategy='most_frequent').fit(X_treino, y_treino)
prev_baseline = baseline.predict(X_teste)

scaler_treinado = modelo.named_steps['standardscaler']
assert np.allclose(scaler_treinado.mean_, X_treino.mean(axis=0))
assert len(y_treino) + len(y_teste) == len(y)
print('Acurácia baseline:', round(accuracy_score(y_teste, prev_baseline), 3))
```

```
print('Acurácia pipeline:', round(accuracy_score(y_teste, previsoes), 3))
print('Acurácia balanceada:', round(balanced_accuracy_score(y_teste, previsoes), 3))
print('Matriz de confusão (linhas=classe real):')
print(confusion_matrix(y_teste, previsoes))
```

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que ajustar StandardScaler antes da separação é inadequado?

Resposta: O scaler usaria estatísticas dos dados de teste e introduziria vazamento de informação.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn. Treinamentos TensorFlow/Keras não foram validados nesta geração editorial.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides ·

TensorFlow: tensorflow.org/guide